EEE4120F

High Performance Embedded Systems

REFERENCE GUIDE

OPEN MPI Basics

Course Team

Course Convener: Associate Professor Simon Winberg

Teaching Assistant: Travimadox Webb

MPI: Message Passing Interface

A Short Reference Guide

1 Introduction

MPI (Message Passing Interface) is a library of C functions, data structures, and macros that enable you to write portable and efficient message-passing programs with ease. This document serves as a starting point for compiling and running your MPI programs.

Tip

Learning Resources:

- Oxford Research Software Engineering Tutorial
- Official OpenMPI Website

2 Verifying MPI Installation

Unlike OpenMP, MPI requires installing external libraries.

2.1 Linux (Ubuntu/Debian)

You need the compiler wrapper (`mpicc`) and the runner (`mpiexec`).

1. **Check Installation:**

```
mpicc --version mpiexec --version
```

2. **If missing (Command not found):**

```
sudo apt update sudo apt install openmpi-bin libopenmpi-dev
```

2.2 Windows (MS-MPI)

Windows does not use `mpicc`. You must rely on Microsoft MPI (MS-MPI).

1. **Check Runtime**: Open PowerShell or Command Prompt.

```
mpiexec
```

If installed, it will display usage options. If it says *"command not recognized"*, you must download **MS-MPI SDK** and **MS-MPI Runtime** from Microsoft.

2. **Check SDK Paths:**

```
set MSMPI
```

This should confirm that the library paths (include/lib) are set.

Warning

Windows Compilation Error: `'mpicc' is not recognized`
Reason: Windows does not have the `mpicc` wrapper.
Fix 1 (Recommended): Use **WSL (Windows Subsystem for Linux)**. This allows you to use standard Linux commands inside Windows.
Fix 2 (Advanced): You must manually link libraries in your compile command:

```
gcc -I"C:\Program Files (x86)\Microsoft SDKs\MPI\Include" ...
```

Tip

Pro Tip for Windows Users: Installing WSL (`wsl --install`) is strongly recommended for this course. It eliminates the configuration headaches associated with MS-MPI and allows you to follow the Linux instructions exactly.

2.3 Timing MPI Programs

To obtain speedup and timing information, use the `MPI_Wtime()` function. It returns a double containing the time in seconds since a reference point.

```
double start = MPI_Wtime();
// ... code to be timed ...
double end = MPI_Wtime();
printf("Total time: %f\n", end - start);
```

Tip

Resolution: The resolution of the timer is 1 microsecond.

3 Compiling MPI Programs

Compiling MPI programs differs significantly between Linux and Native Windows because MPI is a library that requires linking.

3.1 Linux (Ubuntu/Debian) & Windows WSL

On Linux systems (and WSL), we use the `mpicc` wrapper. This is a helper script that automatically adds all the necessary include paths and library flags for you.

1. **Compile:**

```
mpicc my_mpi_code.c -o my_mpi_code
```

2. **Run:** Use `mpirun` to launch the parallel processes.

```
# Launch 4 processes
mpirun -np 4 ./my_mpi_code
```

3.2 Native Windows (MS-MPI with MinGW)

Note: If you are using WSL, follow the Linux instructions above. If you are compiling a native `.exe` using MinGW without WSL, you must manually link the libraries.

1. **The Complex Command:** You must point GCC to the Microsoft SDKs.

```
gcc -I"C:Files (x86)SDKs"
-L"C:Files (x86)SDKs64"
-lmsmpi my_mpi_code.c -o my_mpi_code.exe
```

2. **Run:** Use `mpiexec` (the Windows equivalent of mpirun).

```
mpiexec -n 4 my_mpi_code.exe
```

Tip

Troubleshooting "Undefined Reference":
If you see errors like `undefined reference to 'MPI_Init'`, it means the linker cannot find the MPI library.

- **Linux:** Ensure you used `mpicc`, not `gcc`.
- **Windows:** Ensure you added `-lmsmpi` (dash el ms mpi) to your command.

4 MPI Basics

MPI provides routines allowing processes on different processors/machines to communicate. It is an *interface* specification; different vendors provide optimized implementations for specific architectures. This ensures portability while maintaining performance.

MPI supports two main types of communication:

1. **Point-to-point:** Between 2 processors (one-to-one).
2. **Collective:** One-to-many, many-to-one, or many-to-many.

5 Program Structure

All MPI programs must include the header file `mpi.h`. All MPI functions, variables, and data structures have the prefix "MPI" and return an error code (usually `MPI_SUCCESS`) on completion.

5.1 Initialization and Termination

Before any other MPI routines are called, the program must initialize the environment. Similarly, it must finalize before termination.

```c
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    // ... Application Code ...

    MPI_Finalize();
```

```
    return 0;
}
```

Warning

Process Hang: If `MPI_Finalize()` is not called by all processes, the program may appear to hang.

5.2 Types and Communicators

- **MPI Types:** Most routines require an `MPI_Datatype` argument (e.g., `MPI_CHAR`, `MPI_INT`). You can also create custom types for complex data.
- **Communicators:** A communicator is a group of processes that can communicate amongst themselves.
  - `MPI_COMM_WORLD`: A pre-built communicator containing all processes belonging to the program.
  - **Rank:** Each process has a unique rank within a communicator.

6 Point-to-Point Communication

Point-to-point communication involves a sender using `MPI_Send` and a receiver using `MPI_Recv`. Messages are identified by:

- Sender/Recipient Rank
- Communicator
- Tag (User-defined identifier)
- Data Type and Count

6.1 Send Modes

1. **Synchronous:** Complete only when the receiver acknowledges. Synchronizes sender and receiver.
2. **Buffered:** Data is copied to an internal buffer. Completes when copying is done. No guarantee of receipt.
3. **Ready:** Requires the recipient to have already posted a matching receive. Undefined behavior otherwise.
4. **Standard:** General-purpose. Implementation chooses between synchronous or buffered based on resources.

6.2 Blocking vs Non-Blocking

- **Blocking:** Returns only when the operation completes (safe to reuse memory).
- **Non-Blocking:** Returns immediately. Computation can proceed in parallel with communication. Use `MPI_Wait` or `MPI_Test` to check completion.

Warning

Memory Safety: When using non-blocking calls, do not touch the memory buffer containing the data until the operation is explicitly confirmed complete.

7 Collective Communications

Collective primitives involve multiple processes within a communicator.

- **Barrier (MPI_Barrier):** Blocks until all processes have called the routine. No data transfer.
- **Broadcast (MPI_Bcast):** One process sends data to all others in the communicator.
- **Reduction (MPI_Reduce):** Reduces data spread across processes to a single process (e.g., `MPI_SUM`, `MPI_MAX`).
- **Scatter (MPI_Scatter):** Breaks a contiguous buffer into chunks and passes them to different processes.
- **Gather (MPI_Gather):** Collects data from all processes into a contiguous chunk on the target.

8 API Reference Guide

Below are the syntax and semantics for common MPI functions.

8.1 Environment Management

```
int MPI_Init(int *argc, char ***argv)
```

Returns: Error code.
Must be called before any other MPI routine.

```
int MPI_Finalize()
```

Returns: Error code.
Cleanup routine; all processes must call this before termination.

```
double MPI_Wtime()
```

Returns: Time in seconds since a reference point.

8.2 Communicator Info

```
int MPI_Comm_rank(MPI_Comm comm, int *rank)
```

- `comm` (in): Communicator (e.g., `MPI_COMM_WORLD`).
- `rank` (out): The rank of the calling process.

Allows a process to distinguish itself (e.g., acting as sender or receiver).

```
int MPI_Comm_size(MPI_Comm comm, int *size)
```

- `size` (out): Total number of processes in the communicator.

8.3 Communication

```
int MPI_Send(void *buf, int count, MPI_Datatype dtype,
             int dest, int tag, MPI_Comm comm)
```

This is a blocking, standard-mode send.

- `buf` (in): Initial address of send buffer.
- `count` (in): Number of elements.
- `dest` (in): Rank of destination process.
- `tag` (in): Message identifier.

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm,
             MPI_Status *status)
```

Blocking receive.

- `source` (in): Rank of source (or `MPI_ANY_SOURCE`).
- `status` (out): Information about the received message.

```
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
              int root, MPI_Comm comm)
```

- `buffer` (in/out): Address of buffer.
- `root` (in): Rank of broadcast root.

Data in the root's buffer is sent to all processes.